

DATA ENGINEERING STUDY GUIDE

ETL Explained in Detail

Extract, Transform, Load — the pipelines that move raw data into decisions. Concepts, architecture, tools, real code, and interview prep.



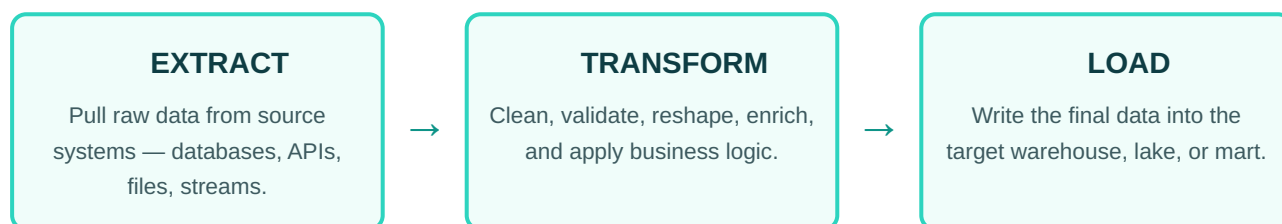
Table of Contents

01 What Is ETL?	3
Why pipelines exist · ETL in the data ecosystem	
02 Extract — Getting the Data	4
Source types · Full vs incremental extraction · CDC	
03 Transform — Shaping the Data	6
Cleaning · Business rules · Enrichment · Aggregation	
04 Load — Delivering the Data	8
Full load · Incremental load · SCD Types	
05 ETL vs ELT	10
06 Batch vs Streaming ETL	11
07 Tools & Ecosystem	12
08 A Worked Example (PySpark)	13
09 Best Practices & Pitfalls	15
10 Interview Q&A	16

01 What Is ETL?

ETL stands for **Extract, Transform, Load** — the process of moving data from one or more source systems into a target system (usually a data warehouse or data lake) while reshaping it along the way so it's clean, consistent, and ready for analysis.

Every organization has data scattered across many places: an operational database that runs the app, a payments API, spreadsheets from finance, log files from servers, a CRM. None of these systems were designed to be queried together. ETL is the engineering discipline that stitches them into one coherent, trustworthy dataset.



WHY IT MATTERS

Without ETL, analysts would query production databases directly — slow, risky, and full of inconsistent formats. ETL centralizes data into a single source of truth that's fast to query and safe to explore.

Where ETL Fits in the Data Stack

Layer	Role	Examples
Source Systems	Where data is created	OLTP databases, SaaS APIs, IoT devices, logs
ETL / ELT	Moves & reshapes data	Airflow, dbt, Spark, Fivetran, SSIS
Storage	Holds integrated data	Data warehouse (Snowflake, BigQuery), data lake (S3, HDFS)
Consumption	Where value is delivered	BI dashboards, ML models, reports

The Medallion Mental Model

Modern ETL pipelines are often organized into layers of increasing refinement — a pattern you've already worked with in the medallion architecture:

- **BRONZE** Raw data, extracted as-is, untouched — the audit trail.
- **SILVER** Cleaned, deduplicated, conformed to a schema — Transform stage output.
- **GOLD** Business-level aggregates and star schemas — ready for BI/Load into marts.

INTERVIEW ANGLE

"Walk me through what ETL means" is a common opener. Give the one-line definition, then immediately connect it to a layer model (bronze/silver/gold) — it signals you think in pipelines, not just definitions.

Extraction is the process of reading data out of source systems without disrupting them. The goal is to get a faithful copy of the source data into a staging area as efficiently as possible.

Common Source Types

Source Type	Examples	Typical Extraction Method
Relational databases	MySQL, SQL Server, PostgreSQL	JDBC/ODBC query, CDC log tailing
APIs	REST, GraphQL, SaaS platforms	Paginated HTTP calls, webhooks
Flat files	CSV, JSON, XML, Parquet	Batch file drop / SFTP / cloud storage
Streams	Kafka topics, IoT sensors	Consumer subscription, continuous read
NoSQL stores	MongoDB, DynamoDB	Native connectors, change streams

Full vs Incremental Extraction

Full Extraction

Pulls the *entire* dataset every run. Simple and safe, but expensive and slow as data grows — fine for small dimension tables, wasteful for a billion-row fact table.

Incremental Extraction

Pulls only what changed since the last run, using a watermark column (`updated_at`, an auto-increment ID) or Change Data Capture. Scales far better.

```
-- Incremental extraction using a watermark column
SELECT *
FROM sales.orders
WHERE updated_at > '2026-08-11 00:00:00' -- last successful run
ORDER BY updated_at;
```

Change Data Capture (CDC)

CDC captures row-level inserts, updates, and deletes directly from a database's transaction log (e.g. MySQL binlog, SQL Server CDC/CT, PostgreSQL WAL) instead of re-querying tables. It's near-real-time and puts almost no load on the source system — the backbone of modern streaming ETL and tools like Debezium.

COMMON PITFALL

Extracting directly from a production OLTP database with heavy queries can lock tables and slow down the app. Best practice: extract from a read replica, or use CDC so the source system barely notices.

Extraction Patterns Compared

Pattern	Latency	Source Load	Complexity
Full batch pull	Hours (scheduled)	High	Low
Incremental (watermark)	Minutes–hours	Medium	Medium
CDC (log-based)	Seconds	Very low	High

03 Transform — Shaping the Data

Transformation is where raw, messy data becomes trustworthy, analysis-ready data. This is usually the largest and most logic-heavy stage of an ETL pipeline.

Core Transformation Categories

1. Cleaning

- Handling **NULLs** — impute, default, or drop depending on business meaning
- Removing **duplicates** from re-extracted or retried loads
- Fixing **data type mismatches** (e.g. string "1,200" → numeric 1200)
- Trimming whitespace, standardizing casing, fixing encoding issues

2. Structural Transformation

- **Normalization / denormalization** to match the target schema
- **Pivoting / unpivoting** rows and columns
- **Joining** multiple sources into one conformed record
- **Splitting** one wide table into fact and dimension tables

3. Business Rules & Enrichment

- Currency conversion, tax calculation, unit standardization
- Deriving new columns (e.g. `profit = revenue - cost`)
- Lookups against reference/dimension tables (e.g. mapping zip → region)
- Masking or hashing PII for compliance

4. Aggregation

- Rolling raw transactions into daily/monthly summaries
- Computing running totals, rankings, moving averages — often via window functions

```
# PySpark: cleaning + enrichment + aggregation in one pipeline
from pyspark.sql import functions as F

silver_df = (bronze_df
    .dropDuplicates(["order_id"])
    .filter(F.col("order_id").isNotNull())
    .withColumn("order_date", F.to_date("order_date"))
    .withColumn("total_amount",
        F.round(F.col("quantity") * F.col("unit_price"), 2))
    .withColumn("amount_flag",
```

```

        F.when(F.col("total_amount") < 0, "REFUND")
          .otherwise("SALE"))
    )

gold_daily_sales = (silver_df
  .groupBy("order_date", "region")
  .agg(F.sum("total_amount").alias("daily_revenue"),
       F.countDistinct("order_id").alias("order_count"))
)

```

REAL PROJECT LINK

This mirrors the bronze → silver → gold pattern you used with `gold.fact_sales` and `gold.dim_customers` — Transform is the step that turns bronze into silver, and silver into gold.

Data Quality Checks Belong in Transform

Check	Example Rule
Completeness	customer_id must not be NULL
Uniqueness	order_id has no duplicates
Validity	email matches a valid format
Referential integrity	every fact row's foreign key exists in the dimension table
Range checks	age between 0 and 120

Loading writes the transformed data into its final destination — a data warehouse table, a data mart, or a lakehouse table — where it becomes queryable by BI tools, analysts, and downstream applications.

Load Strategies

Full Load

Truncate the target table and reload everything. Simple, guarantees consistency, but not scalable for large tables and causes downtime while reloading.

Incremental Load

Insert only new rows, and update/merge changed ones. Efficient, but requires careful handling of updates and deletes (often via `MERGE` /upsert logic).

```
-- Incremental load using MERGE (upsert)
MERGE INTO gold.dim_customers AS target
USING staging.customers_updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
    UPDATE SET target.email = source.email,
               target.updated_at = source.updated_at
WHEN NOT MATCHED THEN
    INSERT (customer_id, email, created_at)
    VALUES (source.customer_id, source.email, source.created_at);
```

Slowly Changing Dimensions (SCD)

A core Load-stage concept: how do you handle a dimension record that changes over time (e.g. a customer moves city)? SCD types define the strategy.

Type	Behavior	Use When
Type 0	Never changes — write once	Immutable attributes (birth date)
Type 1	Overwrite old value, no history kept	Correcting typos, history irrelevant
Type 2	Add a new row, keep old row with an end date	Need full history (most common in warehouses)
Type 3	Add a new column to store the previous value	Only need "current vs. previous"

```
-- Type 2 SCD pattern: expire the old row, insert the new one
UPDATE dim_customers
SET end_date = CURRENT_DATE, is_current = 0
```

```
WHERE customer_id = 1042 AND is_current = 1;

INSERT INTO dim_customers (customer_id, city, start_date, end_date, is_current)
VALUES (1042, 'Cairo', CURRENT_DATE, NULL, 1);
```

INTERVIEW ANGLE

"Explain SCD Type 2" is one of the most-asked data warehousing questions. Answer with the mechanism (expire + insert new row) *and* why it matters: it preserves historical accuracy for reporting.

Load Destinations

- **Data Warehouse** — structured, schema-on-write (Snowflake, Redshift, Synapse, SQL Server)
- **Data Lake** — raw/semi-structured, schema-on-read (S3, ADLS, HDFS with Parquet/ORC)
- **Lakehouse** — combines both worlds (Delta Lake, Apache Iceberg, Hudi)

The order of operations can flip. **ELT** (Extract, Load, Transform) loads raw data into the warehouse first, then transforms it there using the warehouse's own compute power.

	ETL	ELT
Transform happens	Before loading, in a separate engine	After loading, inside the warehouse
Raw data kept?	Often discarded after transform	Always retained (source of truth)
Compute used	Dedicated ETL server/cluster	The warehouse's own engine (elastic)
Best for	Complex logic, legacy systems, on-prem	Cloud warehouses, big data, flexibility
Typical tools	SSIS, Informatica, Talend	Fivetran + dbt, Airbyte + dbt

"ELT became popular once cloud warehouses (Snowflake, BigQuery, Redshift) made storage cheap and compute elastic — it's now cheaper to load everything first and let SQL do the transforming, than to build separate transform infrastructure."

RULE OF THUMB

Modern cloud-native stacks lean ELT. But ETL still dominates when data must be cleaned or masked *before* it ever touches the warehouse — e.g. regulatory or PII-sensitive pipelines.

Batch ETL

Runs on a schedule — hourly, nightly, weekly — processing accumulated data in large chunks. Simple to reason about and debug; the default choice for most reporting pipelines.

Streaming ETL

Processes each event as it arrives, continuously, with latency measured in seconds or milliseconds. Needed for fraud detection, real-time dashboards, and operational alerting.

Aspect	Batch	Streaming
Latency	Minutes to hours	Milliseconds to seconds
Tooling	Airflow + Spark batch jobs, SSIS	Kafka, Spark Structured Streaming, Flink
Complexity	Lower — easier to retry/backfill	Higher — must handle out-of-order events
Cost model	Runs briefly, then idles	Runs continuously

```
# Spark Structured Streaming reading from a Kafka topic
stream_df = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker:9092")
    .option("subscribe", "orders_topic")
    .load())

parsed = stream_df.selectExpr("CAST(value AS STRING)")

query = (parsed.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/chk/orders")
    .start("/delta/bronze/orders"))
```

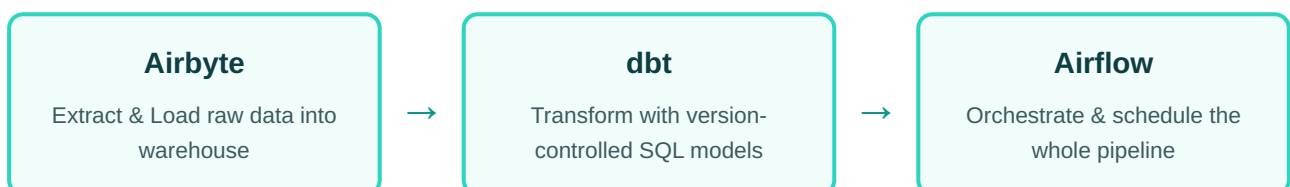
COMMON PITFALL

Teams often reach for streaming because it sounds impressive, when a nightly batch job would be simpler, cheaper, and perfectly adequate. Choose streaming only when the business genuinely needs low latency.

By Category

Category	Tools	Notes
Orchestration	Apache Airflow, Dagster, Prefect	Schedules & monitors pipeline DAGs
Processing engines	Apache Spark, Flink, Pandas	Executes the transform logic at scale
Managed ELT (ingestion)	Fivetran, Airbyte, Stitch	Pre-built connectors, low-code Extract+Load
In-warehouse transform	dbt	SQL-based, version-controlled Transform layer
Streaming	Apache Kafka, Kinesis, Pulsar	Event backbone for real-time ETL
Legacy enterprise ETL	SSIS, Informatica, Talend	GUI-driven, common in on-prem environments
Big data / Hadoop ecosystem	Hive, MapReduce, Sqoop	Batch ETL over HDFS-scale data

A Typical Modern Stack



08

A Worked Example (PySpark)

A compact end-to-end ETL job: extract raw CSV orders, transform with cleaning + business logic, load into a partitioned gold table.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("OrdersETL").getOrCreate()

# ----- EXTRACT -----
bronze_df = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("/data/raw/orders_2026_08_11.csv"))

# ----- TRANSFORM -----
silver_df = (bronze_df
    .dropDuplicates(["order_id"])
    .na.drop(subset=["order_id", "customer_id"])
    .withColumn("order_date", F.to_date("order_date", "yyyy-MM-dd"))
    .withColumn("total_amount", F.round(F.col("qty") * F.col("unit_price"), 2))
    .filter(F.col("total_amount") >= 0)
)

gold_df = (silver_df
    .groupBy("order_date", "region")
    .agg(F.sum("total_amount").alias("revenue"),
        F.count("order_id").alias("orders"))
    .orderBy("order_date")
)

# ----- LOAD -----
(gold_df.write
    .mode("overwrite")
    .partitionBy("order_date")
    .parquet("/data/gold/daily_sales_summary"))
```

REAL PROJECT LINK

This is the same shape as your gold-layer star-schema project on [gold.fact_sales](#) — extract from the bronze source, clean and aggregate in silver/transform logic, then write a partitioned gold table.

09 Best Practices & Pitfalls

Best Practices

- **Idempotency** — re-running a pipeline with the same input should produce the same output, never duplicate rows
- **Incremental by default** — design for watermarks/CDC from day one; full reloads don't scale
- **Schema evolution handling** — expect source schemas to change; fail loudly, don't silently drop columns
- **Observability** — log row counts, null rates, and duration at every stage; alert on anomalies
- **Separate raw from refined** — always keep an immutable bronze/raw copy for reprocessing and audits
- **Version control transforms** — SQL/PySpark logic belongs in git, not ad-hoc notebooks in production

Common Pitfalls

Pitfall	Consequence	Fix
No dedup logic	Duplicate rows on retry	Use natural keys + <code>dropDuplicates</code> /MERGE
Hardcoded dates	Pipeline breaks or silently reprocesses wrong window	Parameterize with execution date
No data quality checks	Bad data reaches dashboards unnoticed	Add validation stage before Load
Tight coupling to source schema	Any source change breaks the whole pipeline	Add schema validation & buffering layer (bronze)

Q1. WHAT'S THE DIFFERENCE BETWEEN ETL AND ELT?

ETL transforms data before loading, using a separate processing engine; ELT loads raw data first and transforms it inside the warehouse using its native compute. ELT dominates modern cloud-warehouse stacks because storage is cheap and warehouse compute is elastic.

Q2. HOW WOULD YOU DESIGN AN INCREMENTAL ETL PIPELINE FOR A TABLE WITH 500M ROWS?

Use a watermark column (or CDC if available) to pull only changed rows, stage them, then MERGE/upsert into the target instead of a full reload. Partition the target table by date to keep writes and scans cheap.

Q3. EXPLAIN SCD TYPE 2 AND WHEN YOU'D USE IT.

Type 2 preserves history by expiring the old dimension row (setting an end_date / is_current flag) and inserting a new row with the updated attribute. Use it whenever historical accuracy matters — e.g. "what region was this customer in when they made this purchase."

Q4. HOW DO YOU MAKE AN ETL JOB IDEMPOTENT?

Design writes so reprocessing the same batch produces the same end state — e.g. overwrite a specific partition rather than append, or MERGE on a natural key rather than blind INSERT.

Q5. BATCH OR STREAMING — HOW DO YOU DECIDE?

Start with the business latency requirement. If stakeholders check numbers once a day, batch is simpler, cheaper, and easier to debug. Reach for streaming only when the use case genuinely needs sub-minute freshness, like fraud detection or live operational dashboards.

Q6. WHAT'S CHANGE DATA CAPTURE AND WHY USE IT OVER POLLING?

CDC reads a database's transaction log to capture row-level changes in near real-time, with minimal load on the source. Polling (re-querying with a watermark) is simpler but adds query load and has higher latency.

— End of Guide · Keep this as a reference alongside your medallion architecture and PySpark transformation notes —